

M11 - Storage & File Formats Foundations

10-lesson learner book | Beginner-first professional institute standard | B01-style teaching contract

Purpose

Choose storage/file formats and reason about Parquet/Delta foundations, partitioning, schema and time travel.

Course boundary

This is the independent Standalone Data Engineer course, not Four-Course C3. Complete the course in sequence; each new idea is still taught from first principles.

Option B institute identity | Charcoal #1F2937 | Teal #14B8A6 | Soft Blue #3B82F6

This book is your teacher

Use one lesson at a time. Do not read protected review before saving a genuine attempt. Every lesson follows the same beginner-first sequence even when the engineering idea becomes more technical.

Daily lesson loop

1) why it matters -> 2) what you learn -> 3) prerequisite -> 4) picture it -> 5) new words -> 6) useful visual/source -> 7) follow once -> 8) expected result -> 9) why it worked -> 10) mistakes -> 11) recovery -> 12) independent task -> 13) validate -> 14) teach it back -> 15) protected review + changed retry -> 16) evidence/mastery + exact next route.

Prerequisite check

Complete M08-M09 ingestion/transformation foundations. You should understand grain, schemas, raw/clean layers, and analytical query patterns.

Attempt protection

A protected review is repair material, not a shortcut. Save the first attempt, open only the matching review if needed, record the repair, close review, then solve a changed retry from a clean state.

Contents / Index

Every formal lesson and its actual page number in this exact rebuilt book.

DE11-01 - Row vs columnar storage; where Parquet fits	5
DE11-02 - Parquet schema, row groups and metadata	7
DE11-03 - Projection, predicates and analytical reads	9
DE11-04 - Compression, small files and compaction	11
DE11-05 - Partitioning strategy and over-partitioning	13
DE11-06 - Lake vs warehouse vs lakehouse	15
DE11-07 - Delta tables, transaction log and ACID concepts	17
DE11-08 - MERGE/upsert and current-state records	19
DE11-09 - Schema enforcement, controlled evolution and compatibility	21
DE11-10 - Time travel, retention, VACUUM safety and medallion checkpoint	23

Prerequisite, tool and outcome map

ASSUMED

Complete M08-M09 ingestion/transformation foundations. You should understand grain, schemas, raw/clean layers, and analytical query patterns.

MODULE PURPOSE

Choose storage/file formats and reason about Parquet/Delta foundations, partitioning, schema and time travel.

EXIT EVIDENCE

Module practical + fresh retest + saved evidence; no critical unresolved failure.

Protected study route

Teaching -> assigned visual/source when useful -> follow once -> close worked material -> independent attempt -> validate -> save evidence -> teach it back. If weak: matching protected review -> targeted repair -> close review -> changed fresh retry -> revalidate.

DE11-01 - Row vs columnar storage; where Parquet fits

1. Why it matters

Explain why analytical systems prefer columnar storage, identify where Parquet fits, and prove the difference between logical table grain and physical file layout. This matters because storage and file-format choices affect schema fidelity, scan behavior and tool compatibility, not just file size.

2. What you are learning

Explain why analytical systems prefer columnar storage, identify where Parquet fits, and prove the difference between logical table grain and physical file layout.

3. Prerequisite / what you already know

Complete M08-M09 ingestion/transformation foundations. You should understand grain, schemas, raw/clean layers, and analytical query patterns.

4. Picture it in your head

WHERE is a gate in front of the result set: each row is tested against the condition and only rows that evaluate TRUE pass through.

5. New words before we use them

A table is a logical collection of rows, but a storage format decides how those values are laid out on disk. Row-oriented layouts keep the fields for one record near each other, which is convenient when applications frequently read or update complete records. Columnar formats group values from the same column together.

6. Useful video / source

Source: Databricks Academy — free data/lakehouse training

Assigned scope: Choose the matching storage / Parquet / Delta module assigned by Mentor

How to use it: Use the matching free training unit for the visual explanation, then return to the Teaching Book and execute the local course lab.

OPEN VIDEO / SOURCE - CLICK HERE (docs.databricks.com)

URL: <https://docs.databricks.com/aws/en/getting-started/free-training>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Use orders.csv. State its grain: one row per order. Compare a question that needs every field for O1008 with a question that needs only region and amount for all orders. The first resembles row retrieval; the second is the kind of scan where columnar storage can help. Run `parquet_probe.py` on the bundled Apache Parquet fixture. Confirm PAR1 at both ends. Then install PyArrow 25.0.* on your learner machine and rerun to expose schema/row-group metadata.

8. Expected result / what you should see

Your explanation must not call Parquet a database. `orders.csv` has 12 rows, quantity 24, amount 2780.00, four regions and four dates.

9. Why it worked

The query is trustworthy because the business question, row grain and transformation stages are explicit, then an independent row/key/total check challenges a plausible-but-wrong result. Your independent check is: For Row vs columnar storage; where Parquet fits: Inspect schema/metadata, row count and one known aggregate; for pruning/partitioning, verify the reader can skip irrelevant columns/partitions rather than inferring it from. The proof combines logical row/schema controls with physical metadata/read behavior.

10. Common beginner mistakes

If row-vs-columnar still feels abstract, draw six columns and four rows on paper, then circle the values needed for `SUM(amount) GROUP BY region`. Count how many cells are relevant.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: WHERE is a gate in front of the result set: each row is tested against the condition and only rows that evaluate TRUE pass through. 3. Run the earliest independent control: For Row vs columnar storage; where Parquet fits: Inspect schema/metadata, row count and one known aggregate; for pruning/partitioning, verify the reader can skip irrelevant columns/partitions rather than inferring it from folder names. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Write a five-sentence explanation that separates logical row grain, columnar storage, Parquet, a query engine, and Delta Lake. Use one concrete analytics query from orders.csv.

13. Validate your result

For Row vs columnar storage; where Parquet fits: Inspect schema/metadata, row count and one known aggregate; for pruning/partitioning, verify the reader can skip irrelevant columns/partitions rather than inferring it from folder names.

14. Teach it back in your own words

Without notes, explain Row vs columnar storage; where Parquet fits in your own words. Start with the purpose: Explain why analytical systems prefer columnar storage, identify where Parquet fits, and prove the difference between logical table grain and physical file layout. Then name one realistic failure mode and explain how you would prove the result using this check: For Row vs columnar storage; where Parquet fits: Inspect schema/metadata, row count and one known aggregate; for pruning/partitioning, verify the reader can skip irrelevant columns/partitions rather than inferring it from.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Row vs columnar storage; where Parquet fits with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE11-02.

DE11-02 - Parquet schema, row groups and metadata

1. Why it matters

Read Parquet schema/metadata and explain row groups, column chunks, pages and the footer without confusing them with business partitions. This matters because storage and file-format choices affect schema fidelity, scan behavior and tool compatibility, not just file size.

2. What you are learning

Read Parquet schema/metadata and explain row groups, column chunks, pages and the footer without confusing them with business partitions.

3. Prerequisite / what you already know

Prerequisite: DE11-01 mastered with saved evidence.

4. Picture it in your head

A Parquet file is organized into row groups with column chunks plus metadata. The metadata lets readers understand schema/statistics without scanning every value.

5. New words before we use them

A Parquet file is not one undifferentiated block. Rows are divided into row groups. Within a row group, each column is stored as a column chunk; chunks contain encoded/compressed pages.

6. Useful video / source

Source: Databricks Academy — free data/lakehouse training

Assigned scope: Choose the matching storage / Parquet / Delta module assigned by Mentor

How to use it: Use the matching free training unit for the visual explanation, then return to the Teaching Book and execute the local course lab.

OPEN VIDEO / SOURCE - CLICK HERE (docs.databricks.com)

URL: <https://docs.databricks.com/aws/en/getting-started/free-training>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

The pure-Python probe reads the final four-byte PAR1 marker and the four-byte little-endian footer-length field immediately before it. It does not decode Thrift metadata;

with PyArrow installed, ParquetFile exposes row-group count, total rows and schema safely. Use the bundled Apache fixture. Record file size, footer metadata length, row-group count and schema when PyArrow is available. Point to the difference between physical row groups and your region directories from the layout lab.

8. Expected result / what you should see

For 12 rows written with a row-group target of 4, expect 3 row groups in this small controlled file. Do not generalize this exact count to production writers.

9. Why it worked

The result is trustworthy because structure is treated as a contract rather than inferred casually; the validation checks fields/types/constraints against the declared shape. Your independent check is: For Parquet schema, row groups and metadata: Query the database/schema context, inspect the created object/constraint, and run one positive plus one deliberately invalid case to prove the rule is actually enforced. The proof combines logical row/schema controls with physical metadata/read behavior.

10. Common beginner mistakes

If PyArrow import fails, verify the active interpreter and run ``python -m pip install "pyarrow==25.0.*"``; use the PAR1 probe meanwhile. Do not install Spark to solve a PyArrow problem.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: A Parquet file is organized into row groups with column chunks plus metadata. 3. Run the earliest independent control: For Parquet schema, row groups and metadata: Query the database/schema context, inspect the created object/constraint, and run one positive plus one deliberately invalid case to prove the rule is actually enforced. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Create ``orders_from_course.parquet`` using `build_course_parquet.py`, which writes `row_group_size=4`. Predict the approximate row-group count before running, then verify it.

13. Validate your result

For Parquet schema, row groups and metadata: Query the database/schema context, inspect the created object/constraint, and run one positive plus one deliberately invalid case to prove the rule is actually enforced.

14. Teach it back in your own words

Without notes, explain Parquet schema, row groups and metadata in your own words. Start with the purpose: Read Parquet schema/metadata and explain row groups, column chunks, pages and the footer without confusing them with business partitions. Then name one realistic failure mode and explain how you would prove the result using this check: For Parquet schema, row groups and metadata: Query the database/schema context, inspect the created object/constraint, and run one positive plus one deliberately invalid case to prove the rule is actually enforced.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Parquet schema, row groups and metadata with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE11-03.

DE11-03 - Projection, predicates and analytical reads

1. Why it matters

Use projection and predicates deliberately, and explain how metadata/statistics may reduce physical reading without promising impossible zero-I/O behavior. This matters because storage and file-format choices affect schema fidelity, scan behavior and tool compatibility, not just file size.

2. What you are learning

Use projection and predicates deliberately, and explain how metadata/statistics may reduce physical reading without promising impossible zero-I/O behavior.

3. Prerequisite / what you already know

Prerequisite: DE11-02 mastered with saved evidence.

4. Picture it in your head

Projection means read only needed columns; predicate pushdown means skip data that cannot satisfy the filter. Picture the reader avoiding unopened boxes rather than scanning everything.

5. New words before we use them

Projection means requesting only the columns a query needs. Predicate filtering means expressing row conditions such as `region = East` or `amount >= 250`. Columnar storage makes projection powerful because an engine can avoid decoding unrelated columns.

6. Useful video / source

Source: Databricks Academy — free data/lakehouse training

Assigned scope: Choose the matching storage / Parquet / Delta module assigned by Mentor

How to use it: Use the matching free training unit for the visual explanation, then return to the Teaching Book and execute the local course lab.

OPEN VIDEO / SOURCE - CLICK HERE (docs.databricks.com)

URL: <https://docs.databricks.com/aws/en/getting-started/free-training>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

With PyArrow, read only `region` and `amount` from the course Parquet, then filter `amount >= 250`. Compare the logical result with the same condition evaluated from `orders.csv`. Record selected columns, qualifying order IDs, qualifying row count and `sum(amount)`. Keep a separate full-scan calculation as a correctness control.

8. Expected result / what you should see

For `amount >= 250` on the course data, qualifying orders are O1002, O1004, O1008, O1010, O1012; count 5 and amount 1610.00.

9. Why it worked

This works because use projection and predicates deliberately, and explain how metadata/statistics may reduce physical reading without promising impossible zero-I/O behavior is made explicit before the result is accepted. Your independent check is: For Projection, predicates and analytical reads: record the actual before/after state and verify the lesson requirement with an independent control. The proof combines logical row/schema controls with physical metadata/read behavior.

10. Common beginner mistakes

If a filtered read returns unexpected rows, prove the predicate against CSV first. Separate a logic error from a storage/engine optimization issue.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Projection means read only needed columns; predicate pushdown means skip data that cannot satisfy the filter. 3. Run the earliest independent control: For Projection, predicates and analytical reads: record the actual before/after state and verify the lesson requirement with an independent control. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Choose another two-column projection and one predicate. Explain which physical work an engine might skip and which result control proves correctness regardless of optimization.

13. Validate your result

For Projection, predicates and analytical reads: record the actual before/after state and verify the lesson requirement with an independent control. Challenge this expected outcome directly: For amount ≥ 250 on the course data, qualifying orders are O1002, O1004, O1008, O1010, O1012; count 5 and amount 1610.00.

14. Teach it back in your own words

Without notes, explain Projection, predicates and analytical reads in your own words. Start with the purpose: Use projection and predicates deliberately, and explain how metadata/statistics may reduce physical reading without promising impossible zero-I/O behavior. Then name one realistic failure mode and explain how you would prove the result using this check: For Projection, predicates and analytical reads: record the actual before/after state and verify the lesson requirement with an independent control.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Projection, predicates and analytical reads with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE11-04.

DE11-04 - Compression, small files and compaction

1. Why it matters

Explain compression, diagnose the small-file problem, and design compaction without confusing fewer files with fewer logical rows. This matters because storage and file-format choices affect schema fidelity, scan behavior and tool compatibility, not just file size.

2. What you are learning

Explain compression, diagnose the small-file problem, and design compaction without confusing fewer files with fewer logical rows.

3. Prerequisite / what you already know

Prerequisite: DE11-03 mastered with saved evidence.

4. Picture it in your head

Many tiny files are like thousands of tiny parcels: metadata/open costs dominate. Compaction combines them into healthier file sizes while preserving table semantics.

5. New words before we use them

Compression reduces bytes, while compaction reduces file count by combining many small files into fewer larger files. They are related but not identical. Parquet commonly compresses within column chunks; repeated values often compress well.

6. Useful video / source

Source: Databricks Academy — free data/lakehouse training

Assigned scope: Choose the matching storage / Parquet / Delta module assigned by Mentor

How to use it: Use the matching free training unit for the visual explanation, then return to the Teaching Book and execute the local course lab.

OPEN VIDEO / SOURCE - CLICK HERE (docs.databricks.com)

URL: <https://docs.databricks.com/aws/en/getting-started/free-training>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Run `layout_lab.py`. It deliberately creates 12 one-row CSV files and a single compacted.csv from the same 12 rows. This is a file-count demonstration only, not a claim that CSV compaction equals Delta OPTIMIZE. Reconcile tiny-file rows to compacted rows and the original controls. Then inspect Parquet compression choices in current PyArrow/Databricks references.

8. Expected result / what you should see

The classroom layout has 12 tiny files and 1 compacted file while preserving the 12-row business dataset. Normal amount remains 2780.00.

9. Why it worked

The setup action is trustworthy because the actual file/system state is checked after the operation; successful clicking or command execution alone is not treated as proof. Your independent check is: For Compression, small files and compaction: Verify the exact path, filename/extension, byte size or hash where appropriate, and confirm the original/working-copy rule was followed. The proof combines logical row/schema controls with physical metadata/read behavior.

10. Common beginner mistakes

If you think “one huge file is always best,” list concurrency, parallelism and rewrite-cost trade-offs. The goal is healthy file sizing, not minimum file count.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Many tiny files are like thousands of tiny parcels: metadata/open costs dominate. 3. Run the earliest independent control: For Compression, small files and compaction: Verify the exact path, filename/extension, byte size or hash where appropriate, and confirm the original/working-copy rule was followed. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Write a production decision note: when would you compact, what would you measure, and what controls must remain invariant? Include one risk of compacting too aggressively.

13. Validate your result

For Compression, small files and compaction: Verify the exact path, filename/extension, byte size or hash where appropriate, and confirm the original/working-copy rule was followed.

14. Teach it back in your own words

Without notes, explain Compression, small files and compaction in your own words. Start with the purpose: Explain compression, diagnose the small-file problem, and design compaction without confusing fewer files with fewer logical rows. Then name one realistic failure mode and explain how you would prove the result using this check: For Compression, small files and compaction: Verify the exact path, filename/extension, byte size or hash where appropriate, and confirm the original/working-copy rule was followed.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Compression, small files and compaction with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE11-05.

DE11-05 - Partitioning strategy and over-partitioning

1. Why it matters

Choose a partitioning strategy from query patterns and cardinality, and identify over-partitioning before it creates tiny-file/metadata problems. This matters because storage and file-format choices affect schema fidelity, scan behavior and tool compatibility, not just file size.

2. What you are learning

Choose a partitioning strategy from query patterns and cardinality, and identify over-partitioning before it creates tiny-file/metadata problems.

3. Prerequisite / what you already know

Prerequisite: DE11-04 mastered with saved evidence.

4. Picture it in your head

A partition is a directory-level pruning clue. Good partition keys remove large irrelevant slices; too many tiny partitions create metadata and small-file overhead.

5. New words before we use them

Dataset partitioning places records into directory/file groups based on a partition key, often a low-to-moderate-cardinality field used for pruning. Good partitioning reduces the candidate files for common queries. Bad partitioning can create excessive directories and tiny files.

6. Useful video / source

Source: Databricks Academy — free data/lakehouse training

Assigned scope: Choose the matching storage / Parquet / Delta module assigned by Mentor

How to use it: Use the matching free training unit for the visual explanation, then return to the Teaching Book and execute the local course lab.

OPEN VIDEO / SOURCE - CLICK HERE (docs.databricks.com)

URL: <https://docs.databricks.com/aws/en/getting-started/free-training>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

The layout lab partitions 12 orders into four region directories. A hypothetical East-only query has one candidate region file instead of four region files. That does not prove Parquet row-group skipping; it demonstrates directory pruning. Compare region, order_date and order_id as candidate partition keys. For each, calculate distinct values on the 12-row data and explain expected file-count pressure.

8. Expected result / what you should see

Course data distinct counts: region=4, order_date=4, order_id=12. A partition-by-order_id plan creates the clearest over-partitioning warning here.

9. Why it worked

The distributed result is trustworthy because logical row/grain rules are kept separate from physical partitions/execution strategy, then reconciled with deterministic counts/schema/output checks. Your independent check is: For Partitioning strategy and over-partitioning: Check schema and deterministic row/key/totals first, then inspect partitions/plan/shuffle evidence separately so performance tuning cannot hide a correctness defect. The proof combines logical row/schema controls with physical metadata/read behavior.

10. Common beginner mistakes

If you cannot choose a partition key, start from the most common selective query boundaries and data volume. Never choose only because a column "looks categorical."

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: A partition is a directory-level pruning clue. 3. Run the earliest independent control: For Partitioning strategy and over-partitioning: Check schema and deterministic row/key/totals first, then inspect partitions/plan/shuffle evidence separately so performance tuning cannot hide a correctness defect. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Design partitioning for a 500M-row event table with frequent last-7-day queries and a very high-cardinality customer_id. Defend your first choice and name the metric that would make you revisit it.

13. Validate your result

For Partitioning strategy and over-partitioning: Check schema and deterministic row/key/totals first, then inspect partitions/plan/shuffle evidence separately so performance tuning cannot hide a correctness defect.

14. Teach it back in your own words

Without notes, explain Partitioning strategy and over-partitioning in your own words. Start with the purpose: Choose a partitioning strategy from query patterns and cardinality, and identify over-partitioning before it creates tiny-file/metadata problems. Then name one realistic failure mode and explain how you would prove the result using this check: For Partitioning strategy and over-partitioning: Check schema and deterministic row/key/totals first, then inspect partitions/plan/shuffle evidence separately so performance tuning cannot hide a correctness defect.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Partitioning strategy and over-partitioning with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE11-06.

DE11-06 - Lake vs warehouse vs lakehouse

1. Why it matters

Compare data lake, data warehouse and lakehouse by storage, governance, transaction and workload needs without treating one architecture as universally best. This matters because storage and file-format choices affect schema fidelity, scan behavior and tool compatibility, not just file size.

2. What you are learning

Compare data lake, data warehouse and lakehouse by storage, governance, transaction and workload needs without treating one architecture as universally best.

3. Prerequisite / what you already know

Prerequisite: DE11-05 mastered with saved evidence.

4. Picture it in your head

A lake emphasizes flexible files, a warehouse emphasizes governed relational analytics, and a lakehouse adds table/transaction semantics over lake-style storage.

5. New words before we use them

A data warehouse traditionally emphasizes managed structured tables, SQL analytics, governance and optimized query serving. A data lake emphasizes low-cost scalable object storage for many data types, but a plain collection of files does not automatically provide table transactions or strong governance. A lakehouse combines open/scalable lake storage with table-management capabilities such as reliable transactions, schemas, metadata and SQL-friendly access.

6. Useful video / source

Source: Databricks Academy — free data/lakehouse training

Assigned scope: Choose the matching storage / Parquet / Delta module assigned by Mentor

How to use it: Use the matching free training unit for the visual explanation, then return to the Teaching Book and execute the local course lab.

OPEN VIDEO / SOURCE - CLICK HERE (docs.databricks.com)

URL: <https://docs.databricks.com/aws/en/getting-started/free-training>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Place the M11 retail data into three conceptual designs: raw object files, curated warehouse tables, and lakehouse tables over object storage. For each, state where transaction/table metadata lives and who enforces contracts. Complete an architecture matrix for storage cost, schema governance, BI serving, ML/file access, transaction support and operational complexity.

8. Expected result / what you should see

Your answer must name trade-offs. "Lakehouse is newer so always better" is an automatic reasoning defect.

9. Why it worked

The table operation is trustworthy because the transaction log defines one committed table state; version/history and row controls prove the logical result rather than trusting physical files alone. Your independent check is: For Lake vs warehouse vs lakehouse: Check current row/key/totals plus Delta history/version. The proof combines logical row/schema controls with physical metadata/read behavior.

10. Common beginner mistakes

If the terms blur together, separate physical storage (files/object store) from table-management/catalog/query layers.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: A lake emphasizes flexible files, a warehouse emphasizes governed relational analytics, and a lakehouse adds table/transaction semantics over lake-style storage. 3. Run the earliest independent control: For Lake vs warehouse vs lakehouse: Check current row/key/totals plus Delta history/version. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Given a small company with 50 GB structured sales data and five BI users, argue both for and against introducing a lakehouse now. Make a recommendation based on need, not fashion.

13. Validate your result

For Lake vs warehouse vs lakehouse: Check current row/key/totals plus Delta history/version. For replay or MERGE, run the changed input twice and confirm the second run does not create unintended additional state.

14. Teach it back in your own words

Without notes, explain Lake vs warehouse vs lakehouse in your own words. Start with the purpose: Compare data lake, data warehouse and lakehouse by storage, governance, transaction and workload needs without treating one architecture as universally best. Then name one realistic failure mode and explain how you would prove the result using this check: For Lake vs warehouse vs lakehouse: Check current row/key/totals plus Delta history/version.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Lake vs warehouse vs lakehouse with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE11-07.

DE11-07 - Delta tables, transaction log and ACID concepts

1. Why it matters

Explain Delta table fundamentals: Parquet data files plus a transaction log, atomic commits, consistent table versions, and why direct file mutation is unsafe. This matters because storage and file-format choices affect schema fidelity, scan behavior and tool compatibility, not just file size.

2. What you are learning

Explain Delta table fundamentals: Parquet data files plus a transaction log, atomic commits, consistent table versions, and why direct file mutation is unsafe.

3. Prerequisite / what you already know

Prerequisite: DE11-06 mastered with saved evidence.

4. Picture it in your head

Git has three useful areas: working tree -> staging area -> commit history. status tells where changes are; add selects the snapshot; commit records it.

5. New words before we use them

Delta Lake adds a transaction-log layer around data files. Instead of deciding table state by listing whatever files happen to exist, readers reconstruct a committed table version from the log. A transaction commit records which files/actions become part of the next version.

6. Useful video / source

Source: Databricks Academy — free data/lakehouse training

Assigned scope: Choose the matching storage / Parquet / Delta module assigned by Mentor

How to use it: Use the matching free training unit for the visual explanation, then return to the Teaching Book and execute the local course lab.

OPEN VIDEO / SOURCE - CLICK HERE (docs.databricks.com)

URL: <https://docs.databricks.com/aws/en/getting-started/free-training>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Run `delta_foundation_simulator.py`. It writes version_0/1/2 snapshots plus numbered classroom log records. This is deliberately NOT a Delta engine; it exists to make versioned commit thinking visible before M13. Inspect log versions 0,1,2. Identify operation, row count and schema. Explain why a reader should move from one committed version to the next rather than combine half-written state.

8. Expected result / what you should see

Simulator versions are [0,1,2]. V0 has 4 rows / spend 820.0; V1 has 5 rows / spend 1050.0; V2 adds `segment` without changing row count.

9. Why it worked

The operation is trustworthy because related changes cross the commit boundary together; a before/after state check proves partial updates are not being mistaken for success. Your independent check is: For Delta tables, transaction log and ACID concepts: Capture before-state, execute the transaction, verify committed rows/totals, then test rollback or replay behavior on a changed case. The proof combines logical row/schema controls with physical metadata/read behavior.

10. Common beginner mistakes

If you equate Delta with “Parquet plus a JSON folder,” focus on the protocol: correctness comes from committed log semantics, not merely the file names.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Git has three useful areas: working tree -> staging area -> commit history. 3. Run the earliest independent control: For Delta tables, transaction log and ACID concepts: Capture before-state, execute the transaction, verify committed rows/totals, then test rollback or replay behavior on a changed case. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Draw a failure timeline where new data files exist but the transaction commit is not published. Explain what an atomic table protocol should expose to a reader.

13. Validate your result

For Delta tables, transaction log and ACID concepts: Capture before-state, execute the transaction, verify committed rows/totals, then test rollback or replay behavior on a changed case.

14. Teach it back in your own words

Without notes, explain Delta tables, transaction log and ACID concepts in your own words. Start with the purpose: Explain Delta table fundamentals: Parquet data files plus a transaction log, atomic commits, consistent table versions, and why direct file mutation is unsafe. Then name one realistic failure mode and explain how you would prove the result using this check: For Delta tables, transaction log and ACID concepts: Capture before-state, execute the transaction, verify committed rows/totals, then test rollback or replay behavior on a changed case.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Delta tables, transaction log and ACID concepts with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE11-08.

DE11-08 - MERGE/upsert and current-state records

1. Why it matters

Reason about MERGE/upsert as matched-update plus unmatched-insert logic, prove current-state controls, and detect ambiguous source matches. This matters because storage and file-format choices affect schema fidelity, scan behavior and tool compatibility, not just file size.

2. What you are learning

Reason about MERGE/upsert as matched-update plus unmatched-insert logic, prove current-state controls, and detect ambiguous source matches.

3. Prerequisite / what you already know

Prerequisite: DE11-07 mastered with saved evidence.

4. Picture it in your head

A transaction groups related inserts/updates/deletes into one atomic unit. UPSERT adds “insert if new, update if existing” but still needs a trustworthy conflict key.

5. New words before we use them

An upsert combines update and insert behavior. Delta `MERGE` expresses a match condition between source and target, then applies actions for matched and unmatched rows. The most important engineering work happens before the syntax: define the target grain and business key, then ensure the source does not contain multiple rows that ambiguously match the same target key unless your logic deliberately resolves them.

6. Useful video / source

Source: Databricks Academy — free data/lakehouse training

Assigned scope: Choose the matching storage / Parquet / Delta module assigned by Mentor

How to use it: Use the matching free training unit for the visual explanation, then return to the Teaching Book and execute the local course lab.

OPEN VIDEO / SOURCE - CLICK HERE (docs.databricks.com)

URL: <https://docs.databricks.com/aws/en/getting-started/free-training>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Simulator v0 has C1-C4. Updates change C2 from Gold/350 to Platinum/420 and insert C5/160. After upsert, target has C1-C5 and spend 1050. Write the four controls before running: matched keys, inserted keys, expected target row count, expected total spend. Then compare to simulator report.

8. Expected result / what you should see

Expected simulator MERGE: updated=[C2], inserted=[C5], rows=5, spend=1050.0. Multiple source records for one target key must be resolved before claiming safe current-state logic.

9. Why it worked

The table operation is trustworthy because the transaction log defines one committed table state; version/history and row controls prove the logical result rather than trusting physical files alone. Your independent check is: For MERGE/upsert and current-state records: Check pre/post row counts, key uniqueness on the expected “one” side, unmatched-key counts, and one important total before and after the relationship operation. The proof combines logical row/schema controls with physical metadata/read behavior.

10. Common beginner mistakes

If target rows unexpectedly multiply, stop and test key uniqueness on both target and intended source update grain before changing the MERGE clause.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: A transaction groups related inserts/updates/deletes into one atomic unit. 3. Run the earliest independent control: For MERGE/upsert and current-state records: Check pre/post row counts, key uniqueness on the expected “one” side, unmatched-key counts, and one important total before and after the relationship operation. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Create a paper source with two different updates for C2. Explain why the source is ambiguous and propose one deterministic pre-merge rule using event timestamp + tie-breaker.

13. Validate your result

For MERGE/upsert and current-state records: Check pre/post row counts, key uniqueness on the expected “one” side, unmatched-key counts, and one important total before and after the relationship operation.

14. Teach it back in your own words

Without notes, explain MERGE/upsert and current-state records in your own words. Start with the purpose: Reason about MERGE/upsert as matched-update plus unmatched-insert logic, prove current-state controls, and detect ambiguous source matches. Then name one realistic failure mode and explain how you would prove the result using this check: For MERGE/upsert and current-state records: Check pre/post row counts, key uniqueness on the expected “one” side, unmatched-key counts, and one important total before and after the relationship operation.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for MERGE/upsert and current-state records with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE11-09.

DE11-09 - Schema enforcement, controlled evolution and compatibility

1. Why it matters

Distinguish schema enforcement from schema evolution and classify additive, rename/drop and type changes by compatibility risk. This matters because storage and file-format choices affect schema fidelity, scan behavior and tool compatibility, not just file size.

2. What you are learning

Distinguish schema enforcement from schema evolution and classify additive, rename/drop and type changes by compatibility risk.

3. Prerequisite / what you already know

Prerequisite: DE11-08 mastered with saved evidence.

4. Picture it in your head

Schema drift is the input contract changing under you. Compatible changes can be handled deliberately; incompatible rows belong in quarantine with the exact reason and original payload.

5. New words before we use them

Schema enforcement rejects or blocks writes that violate the table's expected structure or types. Schema evolution is a controlled process for accepting intended structural change. The safest beginner pattern is "fail closed, inspect, then evolve deliberately," not "accept any new schema automatically." Additive nullable columns are often easier to accommodate than incompatible type changes.

6. Useful video / source

Source: Databricks Academy — free data/lakehouse training

Assigned scope: Choose the matching storage / Parquet / Delta module assigned by Mentor

How to use it: Use the matching free training unit for the visual explanation, then return to the Teaching Book and execute the local course lab.

OPEN VIDEO / SOURCE - CLICK HERE ([docs.databricks.com](https://docs.databricks.com/aws/en/getting-started/free-training))

URL: <https://docs.databricks.com/aws/en/getting-started/free-training>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Simulator v2 adds `segment` to the v1 schema. It is a controlled additive change. Existing five rows remain five rows; each gets a deterministic derived segment. Classify four hypothetical changes: add nullable promo_code, rename amount to net_amount, widen INT to DOUBLE, change date to free-text STRING. Write consumer impact and migration evidence for each.

8. Expected result / what you should see

V2 schema is [id,name,tier,spend,segment] and v2_rows remains 5. Row-count stability alone is not enough; verify schema and key/control totals too.

9. Why it worked

The result is trustworthy because structure is treated as a contract rather than inferred casually; the validation checks fields/types/constraints against the declared shape. Your independent check is: For Schema enforcement, controlled evolution and compatibility: Query the database/schema context, inspect the created object/constraint, and run one positive plus one deliberately invalid case to prove the rule is actual. The proof combines logical row/schema controls with physical metadata/read behavior.

10. Common beginner mistakes

If “schema evolution” sounds like permission to ignore drift, rewrite the rule as: detect -> classify -> approve -> migrate -> validate -> monitor.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Schema drift is the input contract changing under you. 3. Run the earliest independent control: For Schema enforcement, controlled evolution and compatibility: Query the database/schema context, inspect the created object/constraint, and run one positive plus one deliberately invalid case to prove the rule is actually enforced. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Design a schema-change approval checklist with at least one producer check, one consumer check, rollback/replay plan and ownership field.

13. Validate your result

For Schema enforcement, controlled evolution and compatibility: Query the database/schema context, inspect the created object/constraint, and run one positive plus one deliberately invalid case to prove the rule is actually enforced.

14. Teach it back in your own words

Without notes, explain Schema enforcement, controlled evolution and compatibility in your own words. Start with the purpose: Distinguish schema enforcement from schema evolution and classify additive, rename/drop and type changes by compatibility risk. Then name one realistic failure mode and explain how you would prove the result using this check: For Schema enforcement, controlled evolution and compatibility: Query the database/schema context, inspect the created object/constraint, and run one positive plus one deliberately invalid case to prove the rule is actual.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Schema enforcement, controlled evolution and compatibility with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE11-10.

DE11-10 - Time travel, retention, VACUUM safety and medallion checkpoint

1. Why it matters

Use version history/time travel conceptually, explain retention/VACUUM risk, and defend a bronze-silver-gold medallion checkpoint without deleting evidence prematurely. This matters because storage and file-format choices affect schema fidelity, scan behavior and tool compatibility, not just file size.

2. What you are learning

Use version history/time travel conceptually, explain retention/VACUUM risk, and defend a bronze-silver-gold medallion checkpoint without deleting evidence prematurely.

3. Prerequisite / what you already know

Prerequisite: DE11-09 mastered with saved evidence.

4. Picture it in your head

A checkpoint is a durable bookmark of progress. Replay safety means you can restart from a known checkpoint without losing data or creating duplicate side effects.

5. New words before we use them

Time travel means querying a prior committed table version or timestamp while its required files/log history are still retained. It is useful for debugging, audit comparison and reproducibility. Retention policies decide how long historical data files remain available.

6. Useful video / source

Source: Databricks Academy — free data/lakehouse training

Assigned scope: Choose the matching storage / Parquet / Delta module assigned by Mentor

How to use it: Use the matching free training unit for the visual explanation, then return to the Teaching Book and execute the local course lab.

OPEN VIDEO / SOURCE - CLICK HERE ([docs.databricks.com](https://docs.databricks.com/aws/en/getting-started/free-training))

URL: <https://docs.databricks.com/aws/en/getting-started/free-training>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Simulator proves time-travel difference: C2 is Gold in v0 and Platinum in current v2. Its VACUUM action is DRY_RUN_ONLY on purpose. No historical file is deleted in this module. Write a retention decision for a daily table with 10-day incident investigations and 7-day late reprocessing. Then map the retail source through bronze/silver/gold with one validation at each boundary.

8. Expected result / what you should see

Time-travel control: v0 C2 tier=Gold; current v2 C2 tier=Platinum. VACUUM remains dry-run only. Your medallion checklist must preserve raw evidence and include row/key/control reconciliation.

9. Why it worked

The workflow is trustworthy because progress/state is explicit and replaying the same input is tested to converge without silent loss or duplicate side effects. Your independent check is: For Time travel, retention, VACUUM safety and medallion checkpoint: Check current row/key/totals plus Delta history/version. The proof combines logical row/schema controls with physical metadata/read behavior.

10. Common beginner mistakes

If time travel and backups seem identical, separate table-version history from an independent backup/disaster-recovery copy. They solve overlapping but different failure cases.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: A checkpoint is a durable bookmark of progress. 3. Run the earliest independent control: For Time travel, retention, VACUUM safety and medallion checkpoint: Check current row/key/totals plus Delta history/version. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Explain why running VACUUM aggressively immediately after every job could damage debugging/recovery. Then design a medallion promotion checklist for an unseen dataset.

13. Validate your result

For Time travel, retention, VACUUM safety and medallion checkpoint: Check current row/key/totals plus Delta history/version. For replay or MERGE, run the changed input twice and confirm the second run does not create unintended additional state.

14. Teach it back in your own words

Without notes, explain Time travel, retention, VACUUM safety and medallion checkpoint in your own words. Start with the purpose: Use version history/time travel conceptually, explain retention/VACUUM risk, and defend a bronze-silver-gold medallion checkpoint without deleting evidence prematurely. Then name one realistic failure mode and explain how you would prove the result using this check: For Time travel, retention, VACUUM safety and medallion checkpoint: Check current row/key/totals plus Delta history/version.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Time travel, retention, VACUUM safety and medallion checkpoint with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: module practical / Gate route after the mastery checklist.